Taller - Sesion 3 - Series de Tiempo y Python IIE - UNAM

Luis E. Ascencio G.
CIMAT
luis.ascencio@cimat.mx

Abstract Este Notebook incluye una introduccion al manejo de Series de Tiempo con Python

```
import numpy as np # Libreria Matematica basica
import pandas as pd # Libreria para manejo, manipulacion y visualizacion de
datos
from pandas import read_excel # funcion para leer archivos de excel


import matplotlib as mpl # Libreria para visualizacion de datos y graficas
import matplotlib.pyplot as plt   # Funcion para graficar
import seaborn as sns             # Libreria para visualizacion de datos
from pandas.plotting import autocorrelation_plot

from statsmodels.tsa.stattools import acf, pacf
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf

from statsmodels.tsa.arima.model import ARIMA
from statsmodels.tsa.seasonal import seasonal_decompose
from statsmodels.tsa.stattools import adfuller
from statsmodels.tsa.arima_process import ArmaProcess

import session_info
```

```
df = pd.read_csv('AirPassengers.csv')
df1 = read_excel('ClayBricks.xls')
df2 = read_excel('Electricity.xls')
df3 = read_excel('MilkProduction.xls')
df4 = read_excel('JapaneseCars.xls')
df5 = read_excel('HouseSales.xls')
```

Simulacion de Series de Tiempo

Definimos la funcion para graficar series de Tiempo

```
def plot_df(df, x, y, title="", xlabel='Fecha', ylabel='Numero de Pasajeros',
colores="", dpi=100):
    plt.figure(figsize=(15,4), dpi=dpi)
    plt.plot(x, y, color=colores)
    plt.gca().set(title=title, xlabel=xlabel, ylabel=ylabel)
    plt.show()
```

Programa para simular modelos MA(q)

```
def Simul_TS_MA(Q,T):
    print("TS MA de Orden:",len(Q))
    t0=np.random.rand(len(Q))
    E=list(t0)
    X=[]
    x=0
    for i in range(T):
        e=np.random.normal(0,1)
        x=e
        for j in range(len(Q)):
            x=x+Q[j]*E[-j-1]
        X.append(x)
        E.append(e)
        x=0
        e=0
    return(X)
```

Programa para simular modelos AR(p)

```
def Simul_TS_AR(P,T):
    print("TS AR de Orden:",len(P))
    t0=np.random.rand(len(P))
    E=np.random.rand(len(P))
    X=list(t0)
    x=0
    for i in range(T):
        x=np.random.normal(0,1)
        for j in range(len(P)):
            x=x+P[j]*X[-j-1]
        X.append(x)
        x=0
    return(X)
```

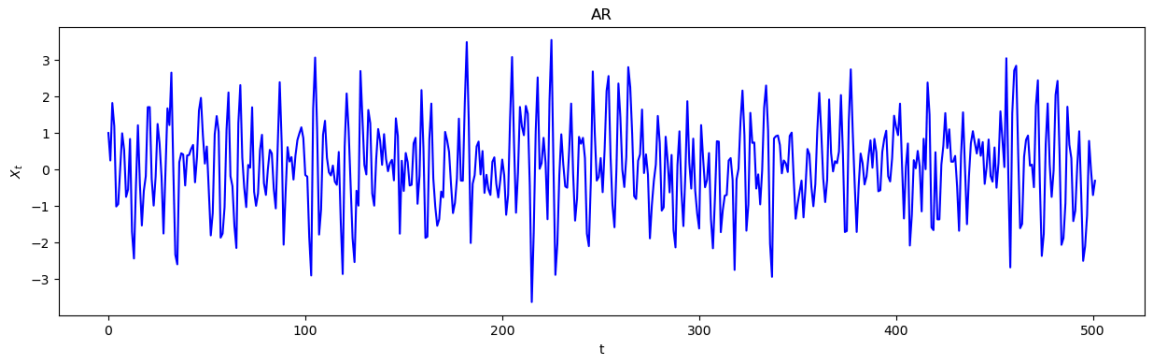
Programa para simular modelos ARMA(p,q)

```
def Simul_TS_ARMA(P,Q,T):
    print("TS ARMA de Orden: p="+str(len(P))+', q='+str(len(Q)))
    t0=np.random.rand(len(P))
    E=np.random.rand(len(P))
    X=list(t0)
    x=0
    for i in range(T):
        e=np.random.normal(0,1)
        x=e
        for j in range(len(P)):
            x=x+P[j]*X[-j-1]
            x=x+Q[j]*E[-j-1]
        X.append(x)
        x=0
    return(X)
```

Simulamos un modelo AR

```
C=[1/2,-1/2]
T=500
AR3_1=Simul_TS_AR(C,T)
plot_df(AR3_1, range(T+len(C)), AR3_1, title="AR", xlabel='t', ylabel='$X_t$',
colores="blue", dpi=100)
```

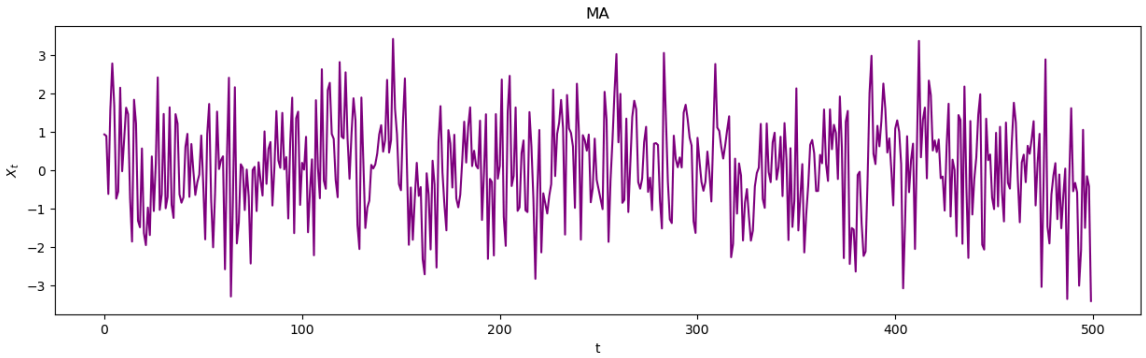
```
TS AR de Orden: 2
```



Simulamos un modelo MA(q)

```
C=[1/2,-1/2,0.3,0.2,0.2,0.3]
MA3_1=Simul_TS_MA(C,T)
plot_df(MA3_1, range(T), MA3_1, title="MA", xlabel='t', ylabel='$X_t$',
colores="purple", dpi=100)
```

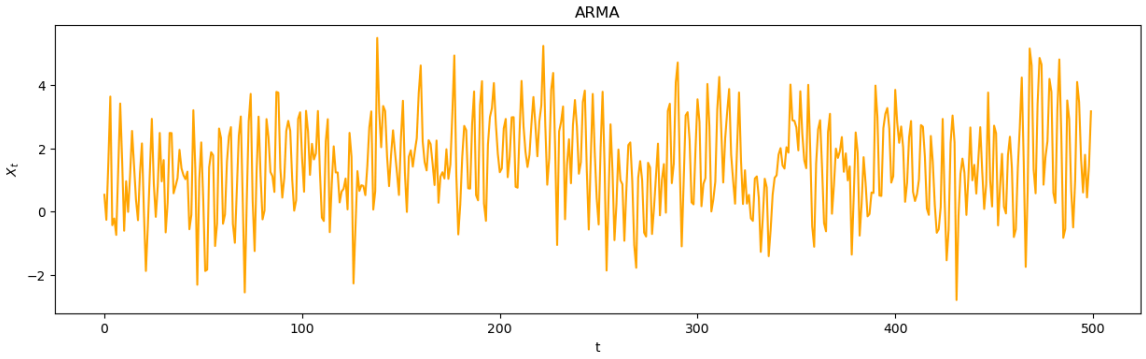
```
TS MA de Orden: 6
```



Simulamos un modelo ARMA(p,q)

```
P=[1/2,-1/2,0.15,0.2,0.2,0.12]
Q=[1/2,-1/2,0.1,0.2,0.2,0.2]
ARMA3_1=Simul_TS_ARMA(P,Q,T)
plot_df(ARMA3_1[len(P):], range(T), ARMA3_1[len(P):], title="ARMA",
xlabel='t', ylabel='$X_t$', colores="orange", dpi=100)
```

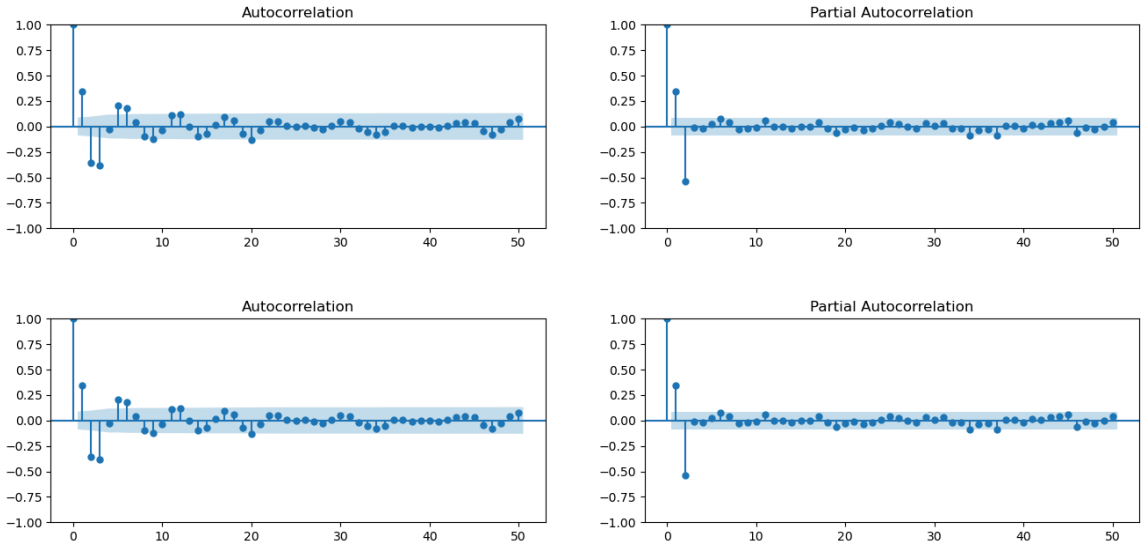
```
TS ARMA de Orden: p=6, q=6
```



Descripcion de modelos mediante ACF y PACF

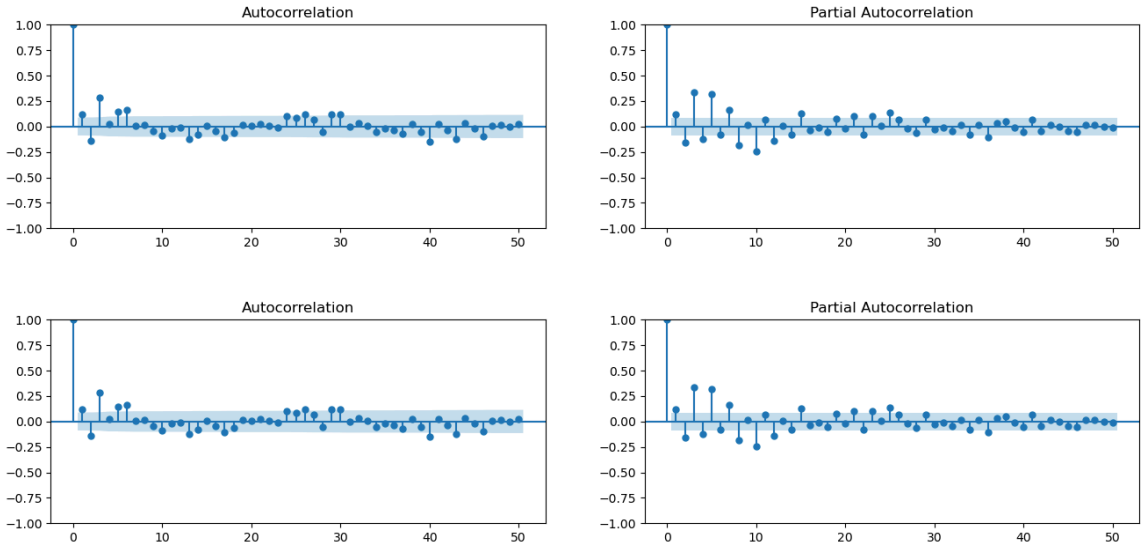
ACF y PACF del modelo AR

```
# Graficas
fig, axes = plt.subplots(1,2,figsize=(16,3), dpi= 100)
plot_acf(AR3_1, lags=50, ax=axes[0])
plot_pacf(AR3_1, lags=50, ax=axes[1])
```



ACF y PACF del modelo MA

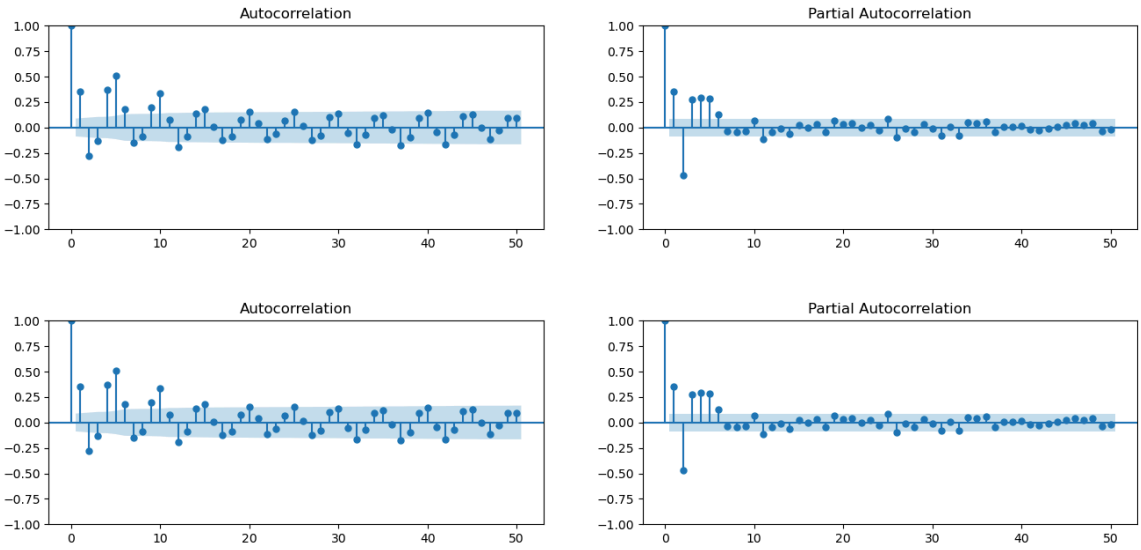
```
# Graficas
fig, axes = plt.subplots(1,2,figsize=(16,3), dpi= 100)
plot_acf(MA3_1, lags=50, ax=axes[0])
plot_pacf(MA3_1, lags=50, ax=axes[1])
```



ACF y PACF del modelo ARMA

```
# Graficas
fig, axes = plt.subplots(1,2,figsize=(16,3), dpi= 100)
```

```
plot_acf(ARMA3_1, lags=50, ax=axes[0])
plot_pacf(ARMA3_1, lags=50, ax=axes[1])
```



Pruebas de Estacionariedad

```
adf1 = adfuller(AR3_1)
print(f'p-value: {adf1[1]}')
```

```
p-value: 0.0
```

```
adf2 = adfuller(MA3_1)
print(f'p-value: {adf2[1]}')
```

```
p-value: 6.078223859765298e-07
```

```
adf3 = adfuller(ARMA3_1)
print(f'p-value: {adf3[1]}')
```

```
p-value: 3.7864347421400604e-05
```

Estacionariedad de los ejemplos de TS

```
Data=[df["#Passengers"],df1["Bricks"],df2["Kwh"],df3["Monthly Milk Production
per Cow"],df4["Price"],df5["HouseSales"]]
```

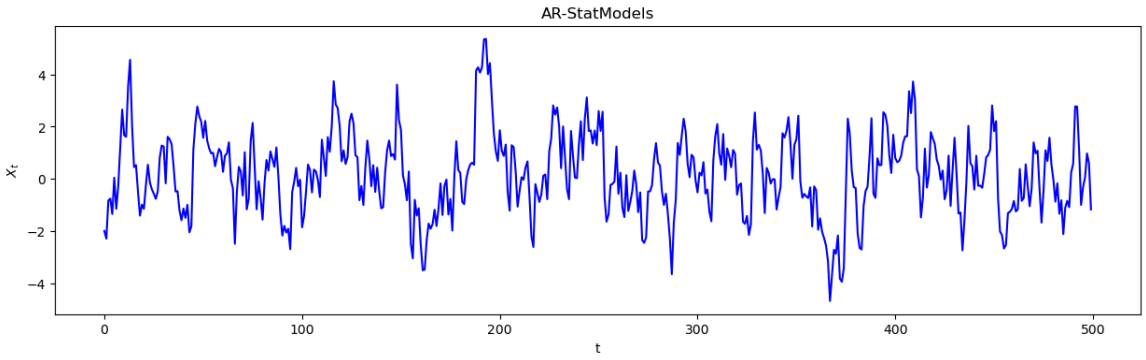
```
w=0
for k in Data:
    w=w+1
    adf = adfuller(k)
    print("P-value de la serie #"+str(w))
    print(f'p-value: {adf[1]}')
```

```
P-value de la serie #1
p-value: 0.991880243437641
P-value de la serie #2
p-value: 0.236826218261678
P-value de la serie #3
p-value: 0.994097902491198
P-value de la serie #4
p-value: 0.6274267086030311
P-value de la serie #5
p-value: 0.16387564674048022
P-value de la serie #6
p-value: 0.03722371625292226
```

Simulacion con StatsModels

```
# Simulate AR(1) process
ar_params = [1, -0.7]  # AR(1) with phi=0.7
ma_params = [1]  # No MA component
ns=500
ar_process = ArmaProcess(ar_params, ma_params)
ar_data = ar_process.generate_sample(nsample=ns)
```

```
plot_df(ar_data,range(ns),ar_data, title="AR-StatModels", xlabel='t',
ylabel='$X_t$', colores="Blue", dpi=100)
```

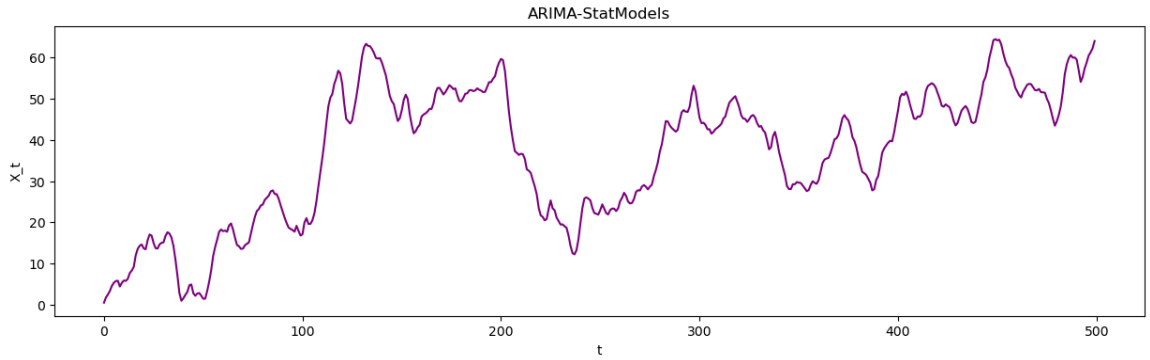


```
# Simulate MA(1) process
ar_params = [1]  # No AR component
ma_params = [1, 0.5]  # MA(1) with theta=0.5
ma_process = ArmaProcess(ar_params, ma_params)
ma_data = ma_process.generate_sample(nsample=ns)
```

```
plot_df(ma_data,range(ns),ma_data, title="MA-StatModels", xlabel='t',
ylabel='$X_t$', colores="purple", dpi=100)
```


```
# Simulate ARIMA(1,1,1) process
ar_params = [1, -0.5]  # AR(1) with phi=0.5
ma_params = [1, 0.4]   # MA(1) with theta=0.4
arima_process = ArmaProcess(ar_params, ma_params)
arima_data = np.cumsum(arima_process.generate_sample(nsample=ns))
```

```
plot_df(arima_data,range(ns),arima_data, title="ARIMA-StatModels", xlabel='t',
ylabel='X_t', colores="purple", dpi=100)
```



Estimacion de Series de Tiempo, Modelos ARIMA

Estimacion modelo AR

```
estim1 = ARIMA(AR3_1, order=(2, 0, 0))
res1 = estim1.fit()
print(res1.summary())
```

```
                               SARIMAX Results
==============================================================================
Dep. Variable:                      y   No. Observations:                  502
Model:                 ARIMA(2, 0, 0)   Log Likelihood                -684.095
Date:                Mon, 27 Oct 2025   AIC                           1376.190
Time:                        05:00:38   BIC                           1393.064
Sample:                             0   HQIC                          1382.810
                                - 502
Covariance Type:                  opg
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
const          0.0899      0.042      2.134      0.033       0.007       0.172
ar.L1          0.5345      0.036     14.836      0.000       0.464       0.605
ar.L2         -0.5417      0.038    -14.228      0.000      -0.616      -0.467
sigma2         0.8922      0.051     17.343      0.000       0.791       0.993
===================================================================================
Ljung-Box (L1) (Q):                   0.01   Jarque-Bera (JB):
5.78
Prob(Q):                              0.94   Prob(JB):
0.06
Heteroskedasticity (H):               0.98   Skew:
0.13
Prob(H) (two-sided):                  0.90   Kurtosis:
3.46
===================================================================================

Warnings:
[1] Covariance matrix calculated using the outer product of gradients
(complex-step).
```

Estimacion modelo MA

```
estim2 = ARIMA(MA3_1, order=(0, 0, 2))
res2 = estim2.fit()
print(res2.summary())
```

```
                               SARIMAX Results
==============================================================================
Dep. Variable:                      y   No. Observations:                  500
Model:                 ARIMA(0, 0, 2)   Log Likelihood                -747.257
Date:                Mon, 27 Oct 2025   AIC                           1502.514
```

```
Time:                        05:00:38   BIC                          1519.373
Sample:                             0   HQIC                         1509.129
                                - 500
Covariance Type:                  opg
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
const          0.0715      0.058      1.238      0.216      -0.042       0.185
ma.L1          0.5939      1.259      0.472      0.637      -1.874       3.061
ma.L2         -0.4061      0.515     -0.789      0.430      -1.415       0.603
sigma2         1.1500      1.453      0.792      0.429      -1.697       3.997
===================================================================================
Ljung-Box (L1) (Q):                   3.00   Jarque-Bera (JB):
5.98
Prob(Q):                              0.08   Prob(JB):
0.05
Heteroskedasticity (H):               1.05   Skew:
-0.19
Prob(H) (two-sided):                  0.77   Kurtosis:
2.63
===================================================================================

Warnings:
[1] Covariance matrix calculated using the outer product of gradients
(complex-step).
```

Estimacion modelo ARMA

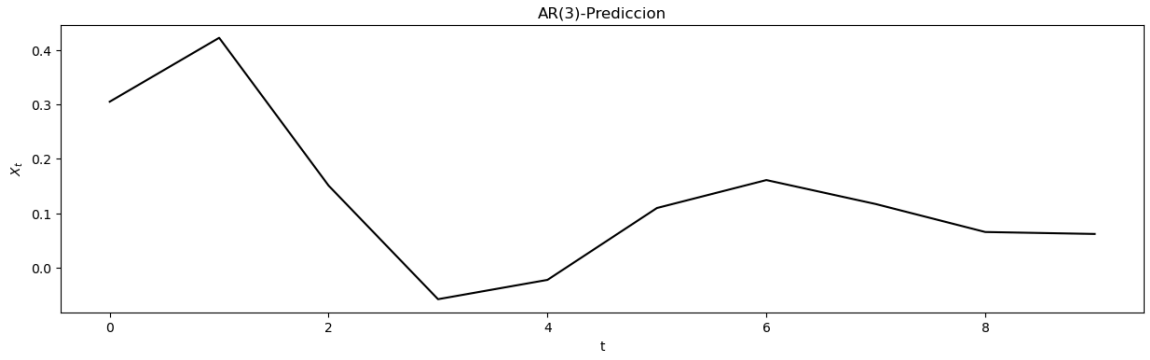
```
estim3 = ARIMA(ARMA3_1, order=(2, 0, 2))
res3 = estim3.fit()
print(res3.summary())
```

```
                               SARIMAX Results
==============================================================================
Dep. Variable:                      y   No. Observations:                  506
Model:                 ARIMA(2, 0, 2)   Log Likelihood                -796.145
Date:                Mon, 27 Oct 2025   AIC                           1604.291
Time:                        05:00:38   BIC                           1629.650
Sample:                             0   HQIC                          1614.237
                                - 506
Covariance Type:                  opg
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
const          1.4682      0.063     23.442      0.000       1.345       1.591
ar.L1          0.2221      0.104      2.141      0.032       0.019       0.425
ar.L2         -0.5386      0.058     -9.237      0.000      -0.653      -0.424
```

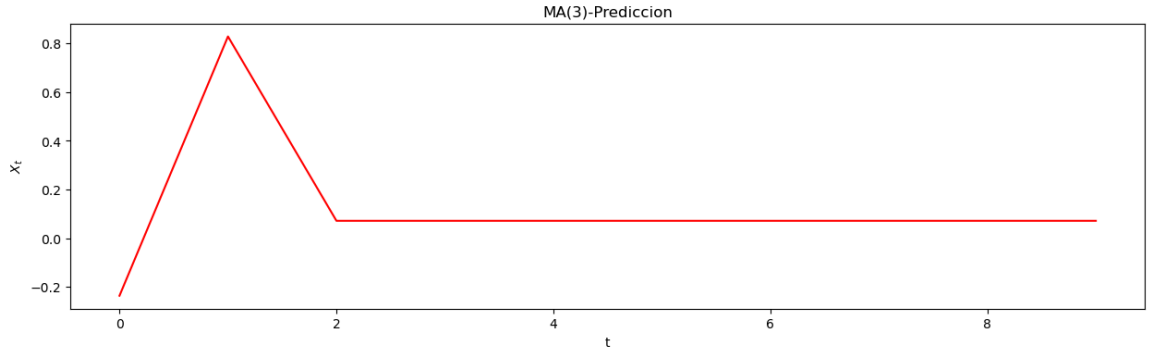
```
ma.L1          0.3600      0.115      3.134      0.002       0.135       0.585
ma.L2          0.2215      0.093      2.382      0.017       0.039       0.404
sigma2         1.3599      0.086     15.854      0.000       1.192       1.528
===================================================================================
Ljung-Box (L1) (Q):                   0.44   Jarque-Bera (JB):
0.01
Prob(Q):                              0.51   Prob(JB):
1.00
Heteroskedasticity (H):               1.02   Skew:
0.01
Prob(H) (two-sided):                  0.92   Kurtosis:
3.01
===================================================================================

Warnings:
[1] Covariance matrix calculated using the outer product of gradients
(complex-step).
```

Prediccion Basica

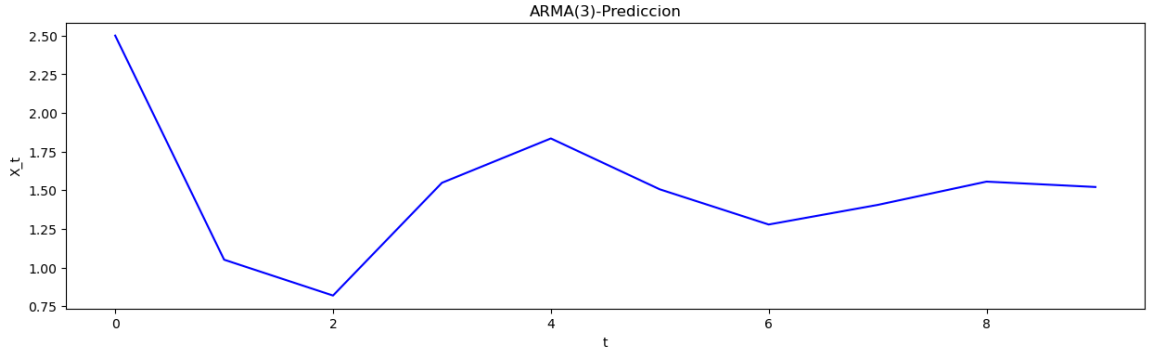
```
tpre=10
plot_df(res1.forecast(steps=tpre), range(tpre),res1.forecast(steps=tpre),
title="AR(3)-Prediccion", xlabel='t', ylabel='$X_t$', colores="black",
dpi=100)
```




```
plot_df(res2.forecast(steps=tpre), range(tpre),res2.forecast(steps=tpre),
title="MA(3)-Prediccion", xlabel='t', ylabel='$X_t$', colores="Red", dpi=100)
```



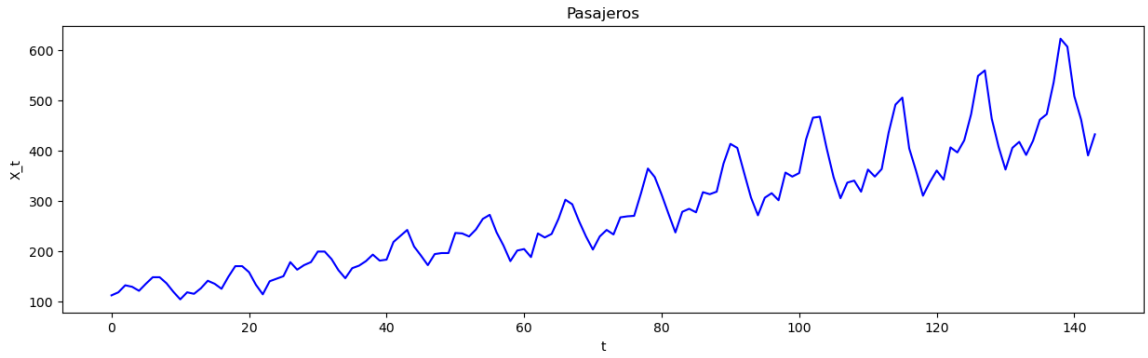

```
plot_df(res3.forecast(steps=tpre), range(tpre),res3.forecast(steps=tpre),
title="ARMA(3)-Prediccion", xlabel='t', ylabel='X_t', colores="Blue", dpi=100)
```



Statsforecast

```
from statsforecast import StatsForecast
from statsforecast.models import AutoARIMA
from statsforecast.utils import AirPassengersDF

dff = AirPassengersDF
```

```
plot_df(dff["y"], range(len(dff)),dff["y"], title="Pasajeros", xlabel='t',
ylabel='X_t', colores="Blue", dpi=100)
```

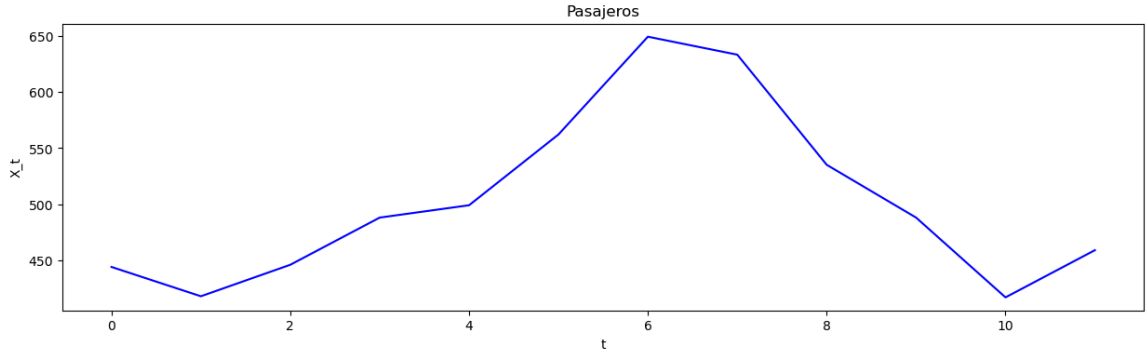



```
sf = StatsForecast(
    models=[AutoARIMA(season_length=12)],
    freq='ME',
)

sf.fit(dff)
dff_forecast=sf.predict(h=12, level=[95])
```

```
dff_forecast
```

	unique_id	ds	AutoARIMA	AutoARIMA-lo-95	AutoARIMA-hi-95
0	1.0	1961-01-31	444.309575	421.279342	467.339808
1	1.0	1961-02-28	418.213738	390.227635	446.199842
2	1.0	1961-03-31	446.243407	412.910032	479.576782
3	1.0	1961-04-30	488.234223	450.621361	525.847084
4	1.0	1961-05-31	499.237066	457.694834	540.779298
5	1.0	1961-06-30	562.236186	517.130961	607.341411
6	1.0	1961-07-31	649.236458	600.822427	697.650489
7	1.0	1961-08-31	633.236374	581.727786	684.744962
8	1.0	1961-09-30	535.236400	480.808289	589.664511
9	1.0	1961-10-31	488.236392	431.037766	545.435017
10	1.0	1961-11-30	417.236394	357.395332	477.077457
11	1.0	1961-12-31	459.236394	396.864759	521.608028

```
plot_df(dff_forecast, range(len(dff_forecast)),dff_forecast["AutoARIMA"],
title="Pasajeros", xlabel='t', ylabel='X_t', colores="Blue", dpi=100)
```




```
session_info.show(html=False)
```

```
-----
matplotlib          3.10.6
numpy               2.3.3
pandas              2.3.3
seaborn             0.13.2
session_info        v1.0.1
statsforecast       2.0.2
statsmodels         0.14.5
-----
IPython             9.6.0
jupyter_client      8.6.3
jupyter_core        5.8.1
jupyterlab          4.4.9
notebook            7.4.7
-----
Python 3.13.5 | packaged by conda-forge | (main, Jun 16 2025, 08:27:50) [GCC
13.3.0]
Linux-6.8.0-85-generic-x86_64-with-glibc2.39
-----
Session information updated at 2025-10-27 05:14
```